

Homework 3: 100 Points

This Homework focusses on Pipelining.

Please submit your answers in a document and upload it in Canvas by the due date.

Please submit by due date, otherwise there is a 10% penalty per day for late submission.

Present your work neat, clear, organized, logical, and make your case; don't leave it to our interpretation of your answer.

1. (65 Points) Consider the code segment in RISC-V:

```
Loop: lw      x1, 0(x2)      ; load X1 from address 0+x2
      addiw   x1, x1, 1      ; x1 = x1 + 1
      sw      x1, 0(x2)      ; store x1 at address 0+x2
      addiw   x2, x2, 4      ; x2 = x2 + 4
      subw    x4, x3, x2      ; x4 = x3 - x2
      bne     x4, x0, -24     ; branch to loop if x4 != 0
```

Assume the initial value of x3 is x2 + 396.

- a) (5 Points) Write the equivalent C/C++ code of the above RISC-V code.
- ```
while (x3 != x2) {
 *x2 = *x2 + 1;
 x2 = x2 + 4;
}
```
- b) (10 Points) Draw the time diagram of the RISC-V code segment for 5-stage RISC pipeline without any forwarding paths but assuming that a register read and write in the same clock cycle. Assume that the comparison is performed in ID stage and PC is updated at the end of ID cycle. Assume that branch is handled by flushing the pipeline.

|          | <u>lw</u> | <u>addiw</u> | <u>sw</u> | <u>addiw x2</u> | <u>subw</u> | <u>bne</u> | <u>flush</u> |
|----------|-----------|--------------|-----------|-----------------|-------------|------------|--------------|
| <u>1</u> | IF        |              |           |                 |             |            |              |
| <u>2</u> | ID        | IF           |           |                 |             |            |              |
| <u>3</u> | EX        | ID           |           |                 |             |            |              |
| <u>4</u> | MEM       | (Stall)      |           |                 |             |            |              |
| <u>5</u> | WB        | EX           | IF        |                 |             |            |              |
| <u>6</u> |           | MEM          | ID        | IF              |             |            |              |

|           |  |    |     |     |     |     |         |
|-----------|--|----|-----|-----|-----|-----|---------|
| <u>7</u>  |  | WB | EX  | ID  | IF  |     |         |
| <u>8</u>  |  |    | MEM | EX  | ID  | IF  |         |
| <u>9</u>  |  |    | WB  | MEM | EX  | ID  |         |
| <u>10</u> |  |    |     | WB  | MEM | EX  |         |
| <u>11</u> |  |    |     |     | WB  | MEM |         |
| <u>12</u> |  |    |     |     |     | WB  |         |
| <u>13</u> |  |    |     |     |     |     | (Flush) |
| <u>14</u> |  |    |     |     |     |     | (Flush) |

- c) (5 Points) How many cycles does the loop take to execute in part b)?  
14 total cycles (12 cycles and 2 flush cycle)
- d) (5 Points) Draw the time diagram of the RISC-V code segment for 5-stage RISC pipeline with full forwarding paths hardware including a register read and write in the same clock cycle. Assume that the comparison is performed in ID stage and PC is updated at the end of ID cycle. Assume that a branch is handled by predicting it as not taken

|           | <u>lw</u> | <u>addiw</u> | <u>sw</u> | <u>addiw x2</u> | <u>subw</u> | <u>bne</u> | <u>flush</u> |
|-----------|-----------|--------------|-----------|-----------------|-------------|------------|--------------|
| <u>1</u>  | IF        |              |           |                 |             |            |              |
| <u>2</u>  | ID        | IF           |           |                 |             |            |              |
| <u>3</u>  | EX        | ID           | IF        |                 |             |            |              |
| <u>4</u>  | MEM       | EX           | ID        | IF              |             |            |              |
| <u>5</u>  | WB        | MEM          | EX        | ID              | IF          |            |              |
| <u>6</u>  |           | WB           | MEM       | EX              | ID          | IF         |              |
| <u>7</u>  |           |              | WB        | MEM             | EX          | ID         |              |
| <u>8</u>  |           |              |           | WB              | MEM         | EX         |              |
| <u>9</u>  |           |              |           |                 | WB          | MEM        |              |
| <u>10</u> |           |              |           |                 |             | WB         |              |
| <u>11</u> |           |              |           |                 |             |            | (Flush)      |

|                  |  |  |  |  |  |  |         |
|------------------|--|--|--|--|--|--|---------|
| <b><u>12</u></b> |  |  |  |  |  |  | (Flush) |
|------------------|--|--|--|--|--|--|---------|

- e) (5 Points) How many cycles does the loop take to execute in part d.  
12 total cycles (10 cycles and 2 flush cycles)
- f) (5 Points) Draw the time diagram as in Part (d) except branches are handled using the delayed branching. Hint: You do not need to schedule an instruction in Delay Slot here; just show delay branching in the time diagram (by one cycle).

|                  | <b><u>lw</u></b> | <b><u>addiw</u></b> | <b><u>sw</u></b> | <b><u>addiw x2</u></b> | <b><u>subw</u></b> | <b><u>bne</u></b> | <b><u>flush</u></b> |
|------------------|------------------|---------------------|------------------|------------------------|--------------------|-------------------|---------------------|
| <b><u>1</u></b>  | IF               |                     |                  |                        |                    |                   |                     |
| <b><u>2</u></b>  | ID               | IF                  |                  |                        |                    |                   |                     |
| <b><u>3</u></b>  | EX               | ID                  | IF               |                        |                    |                   |                     |
| <b><u>4</u></b>  | MEM              | EX                  | ID               | IF                     |                    |                   |                     |
| <b><u>5</u></b>  | WB               | MEM                 | EX               | ID                     | IF                 |                   |                     |
| <b><u>6</u></b>  |                  | WB                  | MEM              | EX                     | ID                 | IF                |                     |
| <b><u>7</u></b>  |                  |                     | WB               | MEM                    | EX                 | ID                |                     |
| <b><u>8</u></b>  |                  |                     |                  | WB                     | MEM                | EX                |                     |
| <b><u>9</u></b>  |                  |                     |                  |                        | WB                 | MEM               |                     |
| <b><u>10</u></b> |                  |                     |                  |                        |                    | WB                | IF                  |
| <b><u>11</u></b> |                  |                     |                  |                        |                    |                   | ID                  |

- g) (5 Points) Based on the part (f) use the scheme of Scheduling the Branch Delay Slot and re-write the RISC-V code to take advantage of the delay slot.

```

Loop: lw x1, 0(x2)
 addiw x1, x1, 1
 sw x1, 0(x2)
 subw x4, x3, x2
 bne x4, x0, Loop
 addiw x2, x2, 4

```

- h) (3 Points) Draw the time Diagram of the RISC-V code in part g

|           | <u>lw</u> | <u>addiw</u> | <u>sw</u> | <u>subw</u> | <u>bne</u> | <u>addiw x2</u> |
|-----------|-----------|--------------|-----------|-------------|------------|-----------------|
| <u>1</u>  | IF        |              |           |             |            |                 |
| <u>2</u>  | ID        | IF           |           |             |            |                 |
| <u>3</u>  | EX        | ID           | IF        |             |            |                 |
| <u>4</u>  | MEM       | EX           | ID        | IF          |            |                 |
| <u>5</u>  | WB        | MEM          | EX        | ID          | IF         |                 |
| <u>6</u>  |           | WB           | MEM       | EX          | ID         | IF              |
| <u>7</u>  |           |              | WB        | MEM         | EX         | ID              |
| <u>8</u>  |           |              |           | WB          | MEM        | EX              |
| <u>9</u>  |           |              |           |             | WB         | MEM             |
| <u>10</u> |           |              |           |             |            | WB              |

- i) (3 Points) Can the Compiler Optimize the code in Part g any further?  
No because there are no more instructions that are available to reduce loop overhead.
- j) (3 Points) Draw the time diagram of the RISC-V code in part i.

|           | <u>lw</u> | <u>addiw</u> | <u>sw</u> | <u>subw</u> | <u>bne</u> | <u>addiw x2</u> |
|-----------|-----------|--------------|-----------|-------------|------------|-----------------|
| <u>1</u>  | IF        |              |           |             |            |                 |
| <u>2</u>  | ID        | IF           |           |             |            |                 |
| <u>3</u>  | EX        | ID           | IF        |             |            |                 |
| <u>4</u>  | MEM       | EX           | ID        | IF          |            |                 |
| <u>5</u>  | WB        | MEM          | EX        | ID          | IF         |                 |
| <u>6</u>  |           | WB           | MEM       | EX          | ID         | IF              |
| <u>7</u>  |           |              | WB        | MEM         | EX         | ID              |
| <u>8</u>  |           |              |           | WB          | MEM        | EX              |
| <u>9</u>  |           |              |           |             | WB         | MEM             |
| <u>10</u> |           |              |           |             |            | WB              |

- k) (3 Points) How many cycles does the code in part j take to execute?  
10 cycles
- l) (3 Points) Compute the speed up obtained in Part k over Parts c and e.  
k = 10 cycles  
c: 14 cycles / 10 cycles = 1.4  
e: 12 cycles / 10 cycles = 1.2
2. (15 Points) Computer C360 is built with no pipelining in single cycle of 7 ns: IF 1 ns, ID 1.5 ns, EX 1 ns, MEM 2 ns, and WB 1.5 ns. Designers consider building C470 using a five stage pipeline based on C360 data.
- a) What is the clock cycle time of the C470 5-stage pipeline?  
The clock cycle time of the C470 5-stage pipeline will be the longest stage time, or **2 ns** (as shown in the MEM cycle)
- b) If there is a stall every four instructions, what is the CPI of C470?  
CPI = 1 (ideal CPI) +  $\frac{1}{4}$  (1 stall for every four instructions) = 1.25
- c) What is the speedup of C470 over C360 assuming Parts b)?  
Speedup: 7 ns / (2 ns \* 1.25) = 2.8
3. (10 Points) Consider the following code segment

```

fld f1, 0(x1)
fld f2, 0(x2)
fmult.d f3, f2, f1
fadd.d f3, f3, 1
addi x3, x3, 1

```

Assume the multi-functional pipeline of Figure C.30. Show the timing diagram of this instruction sequence for the pipeline without any forwarding hardware but assuming a register read and a write in the same clock cycle.

|          | <u>fld f1</u> | <u>fld f2</u> | <u>fmult.d</u> | <u>fadd.d</u> | <u>addi x3</u> |
|----------|---------------|---------------|----------------|---------------|----------------|
| <u>1</u> | IF            |               |                |               |                |
| <u>2</u> | ID            | IF            |                |               |                |
| <u>3</u> | EX            | ID            | IF             |               |                |
| <u>4</u> | MEM           | EX            | ID             |               |                |

|           |    |     |     |     |     |
|-----------|----|-----|-----|-----|-----|
| <u>5</u>  | WB | MEM | EX1 |     |     |
| <u>6</u>  |    | WB  | EX2 | IF  |     |
| <u>7</u>  |    |     | EX3 | ID  |     |
| <u>8</u>  |    |     | EX4 | ID  |     |
| <u>9</u>  |    |     | WB  | ID  |     |
| <u>10</u> |    |     |     | EX1 | IF  |
| <u>11</u> |    |     |     | EX2 | ID  |
| <u>12</u> |    |     |     | EX3 | EX  |
| <u>13</u> |    |     |     | WB  | MEM |
| <u>14</u> |    |     |     |     | WB  |

4. (10 Points) Exercise C.7a (Page C-75 of textbook)

C.7 [10/10] <C.3> In this problem, we will explore how deepening the pipeline affects performance in two ways: faster clock cycle and increased stalls due to data and control hazards. Assume that the original machine is a 5-stage pipeline with a 1 ns clock cycle. The second machine is a 12-stage pipeline with a 0.6 ns clock cycle. The 5-stage pipeline experiences a stall due to a data hazard every five instructions, whereas the 12-stage pipeline experiences three stalls every eight instructions. In addition, branches constitute 20% of the instructions, and the mis-prediction rate for both machines is 5%.

a. [10] <C.3> What is the speedup of the 12-stage pipeline over the 5-stage pipeline, taking into account only data hazards?

**5 Stage Pipeline:**

100 instructions \* (1 stall/5 instructions) = 20 stalls

CPI =  $(100 + 20) / 100 = 1.2$

CPI \* Clock Cycle =  $1.2 * 1 = 1.2$  ns

**12 Stage Pipeline:**

100 instructions \* (3 stalls/8 instructions) = 37.5 stalls

CPI =  $(100 + 37.5) / 100 = 1.375$

CPI \* Clock Cycle =  $1.375 * 0.6 = 0.825$  ns

**Speedup =  $1.2 / 0.825 = 1.45$**